Project 19.1 — DocsGenie-Product-Documentation-Q-And-A-With-Source-Page-Citations

A practice twin of Project 19 — RepoSage. Same skills — RAG over a folder of files where each chunk remembers **where it came from**, a grounded answer that uses *only* the retrieved text, and a citation list you can open — on a new real-world job. **There are no steps here on purpose.** Read the story, build it yourself, and open the hints or solution only if you get stuck.

Type: RAG Q&A over documents · **Stack:** Python, LangChain, Chroma (RAG), OpenAI SDK · **Level:** Intermediate

> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

A support engineer gets the same kind of question all day — "How do I authenticate with the API?", "Which plan includes webhooks?" — and the answer is already written down in the product's documentation. But the docs are spread across a dozen pages, and a general chatbot answers from how products *usually* work, which is often wrong for *this* one and comes with no source to check.

You are going to build **DocsGenie**: ask a question about the product, get an answer grounded in the actual documentation — with the **pages it came from** cited, so the user can jump straight to the source.

- The product's `.md` doc pages are chunked, embedded, and stored in Chroma, each chunk remembering its **page name**.
- A **retriever** pulls the passages closest to the question.
- A deterministic **context builder** stitches them into one block with `# Page:` headers and collects the unique pages for citations.
- The **answer** step has the model answer *only* from that context, and the result names the cited pages.

The one lesson of this project: same as RepoSage — a documentation assistant is RAG with one twist: the "documents" are doc pages, and the **citation is a page name** the user can open. Grounding the answer in retrieved docs (not the model's training memory) is what makes it correct

for *this* product. The context builder and citation list are deterministic, so they test with no model or embeddings.

How it works, from a question to a cited answer (picture the whole machine)

A question comes in. `build_docs_store` has already chunked every doc page and stored each chunk with its page name. The **retriever** pulls the passages closest to the question, each carrying its page. A deterministic **context builder** stitches them into one block with `# Page:` headers, and a **citation builder** collects the unique pages. The **answer** step hands that context to the model with one rule — *answer only from this* — and the pipeline returns both the answer and the list of pages it drew on.

That's the whole machine: **question → retrieve passages → grounded answer + cited pages.**

What you are building (the pieces)

- A RAG layer over the docs folder: `build_docs_store(docs_dir)` (each chunk keeps its page name) and `retrieve_passages(collection, question)`.
- A deterministic `build_context(passages)` (one block, `# Page:` headers) and `cited_pages(passages)` (de-duplicated).
- A grounded `answer_question(model, question, context)` — answers only from the context, says "I don't know" when the docs don't cover it.
- A pipeline `run_query(model, docs_dir, question)` that returns the answer plus the cited pages.

Inputs: a question (and a docs folder). **Outputs:** a grounded answer + the pages it came from. **Rules:** answer only from retrieved docs; "I don't know" when not covered; citations are page names, de-duplicated. The context + citation builders are plain Python, testable with no model.

A real example to build and test against

- A tiny `docs/` folder: `installation.md`, `authentication.md`, `billing.md`.
- "How do I authenticate with the API?" → an answer about API keys, citing `authentication.md`.
- "How do I train a machine-learning model here?" → the docs don't cover it, so the answer admits it doesn't know.

How to practice this

1. Read this story. **Don't open the solution yet.** (Review Project 19 / Doc08 RAG if hazy.)
2. Build the **deterministic parts first** — `build_context` + `cited_pages` — and test them with hand-made passages, no model.
3. Build the RAG layer: the docs store (keep the page name in metadata) and the retriever.
4. Build the grounded `answer_question`, then wire the pipeline: retrieve → context → answer → attach cited pages.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

"Answer a question from our own documents, and cite the page" is the shape of documentation bots, help-center search, internal knowledge assistants, and policy look-ups alike. The skill is the grounding — retrieve real passages, answer only from them, cite the source — and it's reusable anywhere an answer must be correct for *your* material and traceable back to it. Build DocsGenie and you can turn any folder of documents into a grounded, citable assistant.

Done when

- The docs folder is chunked and stored, each chunk keeping its page name.
- The answer uses only the retrieved passages, and says "I don't know" when the docs don't cover the question.
- Every answer names the cited pages, de-duplicated.
- `build_context` + `cited_pages` are deterministic, so the engine tests with no model.
- Your tests pass: the builders (hand-made passages) and a live question answered from the docs with the right page cited.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.